Constructing Custom Equity Indexes Using Python and WRDS/CRSP

Technical Guidance for MSBA FA Students

Michael Sury
Department of Finance
University of Texas at Austin

August 6, 2025

Overview and Motivation

Custom index construction is a foundational skill in empirical finance, enabling researchers and practitioners to develop benchmarks tailored to specific hypotheses, investment mandates, or academic inquiries. Constructing such indices allows for evaluating portfolio strategies, and analyzing the impact of weighting schemes, reconstitution, and rebalancing methodologies. This document presents an example of index construction using Python and WRDS/CRSP data, providing a framework for building equal-, value-, and price-weighted indexes, comparing them to benchmark ETFs, and performing correlation and performance analyses.

1 Project Assignment: Objectives and Specifications

We will explore construction methods via a project assignment. The objective of the assignment is to develop a Python-based program that constructs and analyzes custom equity indices from CRSP data accessed through WRDS. This project should culminate in a script that performs the following key tasks:

Detailed Objectives

- Query, merge, and clean CRSP monthly data, restricting the universe to US common shares on major exchanges.

- Incorporate delisting returns to mitigate survivorship bias, compute effective holding period returns, and use only lagged market cap/prices for index reconstitution and weighting.
- Implement data cleaning and feature engineering procedures, notably computing periodic market capitalization.
- Construct three types of indices using the top N stocks by market capitalization:
  - Equal-weighted index (EW)
  - Value-weighted index (VW)
  - Price-weighted index (PW)
- Incorporate procedures for *reconstitution* at regular intervals (e.g., daily or monthly), selecting the top N stocks anew, and *rebalancing* index weights accordingly. Methodology should be free of look-ahead bias.
- Align custom indices with popular ETF benchmarks (SPY, IWM, QQQ) for comparative analysis.
- Compute log returns and correlation matrices to assess co-movement and diversification characteristics.
- Visualize index levels and correlation matrices with clear, informative graphics.
- Ensure modular, readable code with appropriate documentation sufficient for replication and extension.

Project Specifications

1. Filter equities by security and exchange code: shrcd $\in \{10, 11\}$, exchcd $\in \{1, 2, 3\}$ (US common shares; NYSE, AMEX, NASDAQ).
2. Incorporate delisting returns: effective monthly return $r_{eff} = (1 + r_{CRSP}) \times (1 + r_{delist}) - 1$.
3. Avoid look-ahead bias: constituent selection and index weights use $t - 1$ info, with returns compounded for $t - 1 \to t$.
4. Apply monthly reconstitution and rebalancing at each month-end.
5. Handle missing data by carrying forward previous index level or excluding incomplete periods.
6. Provide options to experiment with turnover, costs, and alternative weighting/divisor methods.

2 CRSP Data and Monthly Panel Construction

2.1 WRDS and Data Querying

Wharton Research Data Services (WRDS) provides SQL-like programmatic access to CRSP. The `wrds` Python library facilitates connecting to WRDS and executing structured queries to retrieve granular time series data efficiently.

```python
import wrds
db = wrds.Connection()
```

Listing 1: Conecting to WRDS

2.2 CRSP and the Investable Universe

The CRSP monthly stock file (MSF) contains essential data for equity index analysis: price (`prc`), shares outstanding (`shrout`), and monthly return (`ret`). Security and exchange codes required for universe filtering (US common shares, major exchanges) are sourced from the name history tables.

```python
def get_crsp_monthly_panel(db: wrds.Connection, start: str, end: str) -> pd.DataFrame:
    """Pull the CRSP monthly panel *with* security descriptors via a time-bounded
    join to msenames. This is necessary because shrcd/exchcd are not columns in
    MSF; they live in the name history tables.

    Filtering choices (my defaults for class):
      - U.S. common stocks only: shrcd in (10, 11)
      - Major exchanges only:    exchcd in (1, 2, 3)  # NYSE, AMEX, NASDAQ

    Returns
    -------
    DataFrame with: date, permno, prc, shrout, ret, and any columns needed for
    market cap construction and return math.
    """
    years = range(pd.to_datetime(start).year, pd.to_datetime(end).year + 1)

    frames: List[pd.DataFrame] = []
    for y in tqdm(years, desc="CRSP Query Progress"):
        chunk_start = f"{y}-01-01" if y > START_TS.year else start
        chunk_end   = f"{y}-12-31" if y < END_TS.year   else end

        # Time-bounded join ensures we take the *correct* security descriptor
        # (shrcd/exchcd) that was in force on each MSF observation date.
        q = f"""
            SELECT m.date, m.permno, m.prc, m.shrout, m.ret
                  , n.shrcd, n.exchcd
            FROM crsp.msf AS m
            JOIN crsp.msenames AS n
              ON  m.permno = n.permno
              AND m.date BETWEEN n.namedt AND COALESCE(n.nameendt, DATE '9999-12-31')
```

```
            WHERE m.date BETWEEN '{chunk_start}' AND '{chunk_end}'
              AND n.shrcd IN (10, 11)      -- U.S. common shares
              AND n.exchcd IN (1, 2, 3)    -- NYSE, AMEX, NASDAQ
              AND m.prc IS NOT NULL
              AND m.shrout IS NOT NULL
            ORDER BY m.date, m.permno;
        """
        frames.append(db.raw_sql(q))

    df = pd.concat(frames, ignore_index=True)

    # Standardize dtypes and construct market cap (for value-weighting and Top-N)
    df['date']   = pd.to_datetime(df['date'])
    df['prc']    = pd.to_numeric(df['prc'], errors='coerce').abs()   # CRSP may store sign with return convention
    df['shrout'] = pd.to_numeric(df['shrout'], errors='coerce') * 1000.0  # in thousands of shares
    df['ret']    = pd.to_numeric(df['ret'], errors='coerce')
    df['mktcap'] = df['prc'] * df['shrout']
    return df
```

Listing 2: Pulling and merging panel data with security descriptors

2.3 Delisting Return Mitigation

One aspect of robust index construction is the proper handling of delisting events, which—if ignored—can materially bias return estimates upward due to survivorship effects. In financial databases like CRSP, firms that are removed from exchanges often have a final payoff in the form of a delisting return (`dlret`), capturing the value lost in bankruptcy, merger cashouts, or other terminal events. Simply dropping such firms from the sample allows failed stocks to "disappear," artificially inflating index performance.

To mitigate this, our methodology merges the monthly stock panel (`msf`) with the delisting return table (`msedelist`), matching on security identifier (`permno`) and call date. The combined "effective return" for each asset-month is then computed as:

$$r_{\text{eff}} = (1 + r_{\text{CRSP}}) \times (1 + r_{\text{delist}}) - 1$$

Here, r_{CRSP} is the standard monthly return (including dividends), and r_{delist} is the delisting return (if applicable; zero otherwise). This formula correctly captures the total return for all securities, ensuring that the economic consequence of each firm's exit—the often large negative return in the delisting period—is reflected in the index outcome.

By rigorously including r_{eff} in all index calculations, the analysis avoids upward bias associated with survivorship and produces results that more accurately represent real-world investable outcomes, especially over long horizons or in turbulent markets when delistings are frequent.

```
def add_effective_returns(panel: pd.DataFrame, delist: pd.DataFrame) -> pd.DataFrame:
```

```
    out = panel.merge(delist, on=['permno', 'date'], how='left')
    out['ret_eff'] = (1.0 + out['ret'].fillna(0.0)) * (1.0 + out['dlret'].fillna(0.0)) -
    1.0
    return out
```

Listing 3: Merging delisting returns and computing effective returns

3 Mathematics and Index Construction

3.1 Market Capitalization and Stock Selection

Market capitalization for security i on day t is given by:

$$\text{mktcap}_{i,t} = |\text{prc}_{i,t}| \times \text{shrout}_{i,t}$$

where price is taken as absolute due to CRSP conventions allowing negative prices for bid-ask adjustment, and shares outstanding are converted from thousands to actual count by multiplication. These adjustments were part of the *get_crsp_monthly_panel* function listed in previous code snippet (listing 2).

At each *reconstitution* date, the top N securities by market capitalization are selected to form the index universe:

$$\mathcal{S}_t = \{\text{permno}_1, \ldots, \text{permno}_N\}$$

This step refreshes the index membership and differs from simple *rebalancing*, where the portfolio weights are adjusted without changing constituent securities.

3.2 Reconstitution vs. Rebalancing

Reconstitution refreshes the index membership at each period using information as of $t-1$. **Rebalancing** adjusts the weights among present constituents based on lagged market cap or prices, but does not alter the set of stocks except at reconstitution.

This logic ensures:

- All index changes reflect only tradable, known information at the end of the previous period.
- Delisting and return handling is correct for accurate empirical and investable performance measurement.
- The methodology avoids look-ahead and survivorship biases, crucial for both robust research and practical investment modeling.

Once again, understanding the distinction is critical:

- **Reconstitution** changes the set of constituent securities.
- **Rebalancing** changes the portfolio weights without altering the set of constituents.

Effective index management requires implementing both carefully, ensuring returns are compounded correctly using prior period weights and that changes in membership structure are handled appropriately.

3.3 Index Weighting Schemes

The three classical weighting schemes used here are defined as follows:

Equal-Weighted (EW):

$$w_{i,t-1}^{(EW)} = \frac{1}{N}$$
$$R_t^{(EW)} = \sum_{i \in \mathcal{S}_t} w_{i,t-1}^{(EW)} r_{i,t}$$

Value-Weighted (VW):

$$w_{i,t-1}^{(VW)} = \frac{\text{mktcap}_{i,t-1}}{\sum_{j \in \mathcal{S}_t} \text{mktcap}_{j,t-1}}$$
$$R_t^{(VW)} = \sum_{i \in \mathcal{S}_t} w_{i,t-1}^{(VW)} r_{i,t}$$

Price-Weighted (PW):

$$w_{i,t-1}^{(PW)} = \frac{|\text{prc}_{i,t-1}|}{\sum_{j \in \mathcal{S}_t} |\text{prc}_{j,t-1}|}$$
$$R_t^{(PW)} = \sum_{i \in \mathcal{S}_t} w_{i,t-1}^{(PW)} r_{i,t}$$

The index levels are then updated by compounding previous levels by these returns:

$$L_t = L_{t-1}(1 + R_t), \quad L_0 = 100$$

3.4 Monthly Index Calculation Loop

At the heart of index construction is an iterative process that advances month-by-month through the sample period, rigorously applying both reconstitution and rebalancing principles using lagged information. The loop implements the following steps for each monthly transition:

- **Identify adjacent months**: For each period t, use information available at $t-1$ (the preceding month) to define the investable universe and compute portfolio weights. This ensures all rebalancing and reconstitution decisions are made based solely on observable, non-forward-looking data, thereby avoiding look-ahead bias.
- **Filter and select Top-N constituents**: At month $t-1$, rank all eligible securities by market capitalization and select the top N as index constituents for month t. Their respective market caps and prices from $t-1$ inform weight assignment.
- **Match holdings and returns**: For continuity, ensure that only securities present in both $t-1$ and t are included in the index calculation for the holding period. Securities that delist between months have their final returns properly incorporated via the ret_eff (effective return), which includes delisting returns.
- **Calculate gross returns for each weighting scheme**: For the chosen set, compute the aggregate index return as a gross multiple:
  1. Equal-weighted: arithmetic mean of each constituent's gross return $(1 + \text{ret_eff})$.
  2. Value-weighted: weighted mean, using t-1 market cap proportions.
  3. Price-weighted: weighted mean, using t-1 price proportions.
- **Compound index levels**: Multiply the previous index level by the gross return to obtain the new level for month t. This is repeated for each weighting scheme, forming complete time series of index evolution.
- **Handle missing or incomplete data**: If data for a month is missing or insufficient (e.g., due to delistings or incomplete records), the index simply carries forward its last computed level, maintaining time series integrity.

This structure not only facilitates precise empirical analysis of index methodologies but also ensures that biases are actively mitigated, and that performance statistics reflect investable, real-world index behavior over the entire sample interval.

```python
def build_topn_indexes(panel: pd.DataFrame, top_n: int) -> pd.DataFrame:
    """Construct Equal-, Value-, and Price-Weighted index levels (base=100).

    Methodology reminder:
      At month t, choose constituents and weights from t-1 information.
      Grow index from t-1 to t using ret\_eff (includes delistings).

```

```python
    Returns a DataFrame with three columns (EW, VW, PW) indexed by month t.
    """
    # Organize by month for quick lookup
    panel = panel.sort_values(['date', 'permno'])
    months = sorted(panel['date'].unique())
    by_month = {d: g for d, g in panel.groupby('date')}

    ew_level = [100.0]
    vw_level = [100.0]
    pw_level = [100.0]

    # Loop from the 2nd available month; we need t-1 and t.  I also use tqdm because I like to see progress bar
    for i in tqdm(range(1, len(months)), desc="Index Calculation Progress"):
        t_1, t = months[i-1], months[i]
        g_t1 = by_month[t_1].dropna(subset=['mktcap', 'prc'])
        g_t  = by_month[t]

        if g_t1.empty or g_t.empty:
            # If either month lacks data, carry forward
            ew_level.append(ew_level[-1])
            vw_level.append(vw_level[-1])
            pw_level.append(pw_level[-1])
            continue

        # Top-N by market cap at t-1 (investable & no look-ahead)
        chosen = g_t1.sort_values('mktcap', ascending=False).head(top_n).set_index('permno')

        # Ensure we have returns at t for the chosen PERMNOs
        g_t = g_t.set_index('permno')
        common = chosen.index.intersection(g_t.index)
        if len(common) == 0:
            ew_level.append(ew_level[-1])
            vw_level.append(vw_level[-1])
            pw_level.append(pw_level[-1])
            continue

        r_t = g_t.loc[common, 'ret_eff'].astype(float)

        # Equal-Weighted: average of gross returns over available names
        r_t_ew = r_t.dropna()
        ew_gross = (1.0 + r_t_ew).mean() if not r_t_ew.empty else 1.0

        # Value-Weighted: weights from t-1 market cap
        w_vw = chosen.loc[common, 'mktcap'].astype(float)
        mask_vw = r_t.notna()
        w_vw = w_vw.loc[mask_vw]
        r_vw = r_t.loc[mask_vw]
        if w_vw.empty or w_vw.sum() <= 0:
            vw_gross = 1.0
        else:
            w_vw = w_vw / w_vw.sum()
            vw_gross = ((1.0 + r_vw) * w_vw).sum()

```

```
        # Price-Weighted: weights from t-1 price (Dow-style approximation)
        w_pw = chosen.loc[common, 'prc'].astype(float)
        mask_pw = r_t.notna()
        w_pw = w_pw.loc[mask_pw]
        r_pw = r_t.loc[mask_pw]
        if w_pw.empty or w_pw.sum() <= 0:
            pw_gross = 1.0
        else:
            w_pw = w_pw / w_pw.sum()
            pw_gross = ((1.0 + r_pw) * w_pw).sum()

        # Compound index levels
        ew_level.append(ew_level[-1] * float(ew_gross))
        vw_level.append(vw_level[-1] * float(vw_gross))
        pw_level.append(pw_level[-1] * float(pw_gross))

    # Convert to tidy Series/DataFrame indexed by month t (growth from t-1 to t)
    idx_months = pd.to_datetime(months[1:])
    ew = pd.Series(ew_level[1:], index=idx_months, name='Equal-Weighted')
    vw = pd.Series(vw_level[1:], index=idx_months, name='Value-Weighted')
    pw = pd.Series(pw_level[1:], index=idx_months, name='Price-Weighted')
    return pd.concat([ew, vw, pw], axis=1)
```

Listing 4: Monthly reconstitution loop for index levels

4 Benchmark ETF Indexes and Log-Return Analytics

ETF series (SPY, IWM, QQQ) are retrieved by matching tickers to appropriate PERMNOs. Index series are compounded using the ETF's monthly returns and compared alongside custom indexes.

```
def get_etf_index(db: wrds.Connection, permno: int, start: str, end: str, label: str) -> pd
    .Series:
    """Build a 100-based index level for an ETF by compounding monthly returns.
    Useful for quick benchmark overlays.
    """
    q = f"""
        SELECT date, ret
        FROM crsp.msf
        WHERE permno = {permno}
          AND date BETWEEN '{start}' AND '{end}'
        ORDER BY date;
    """
    d = db.raw_sql(q)
    d['date'] = pd.to_datetime(d['date'])
    d = d.sort_values('date')
    d['ret'] = pd.to_numeric(d['ret'], errors='coerce').fillna(0.0)
    idx = (1.0 + d['ret']).cumprod() * 100.0
    idx.index = d['date']
    idx.name = label
    return idx
```

Listing 5: ETF index level construction

4.1 Log Return Correlations

Algorithmic alignment on common date grids is performed before computing correlations. Log returns are favored for their additive and symmetric properties.

```
def align_and_corr(levels: List[pd.Series]) -> pd.DataFrame:
    common = set(levels[0].index)
    for s in levels[1:]:
        common &= set(s.index)
    common = sorted(common)
    aligned = [s.loc[common] for s in levels]
    panel = pd.concat(aligned, axis=1)
    log_ret = np.log(panel / panel.shift(1)).dropna(how='any')
    return log_ret.corr()
```

Listing 6: Log returns and correlation matrix calculation

5 Putting it Together

Below is a partial snippet of how some of the functions described above can be put together.

```
# 1) Pull the panel and delistings, then compute effective returns
print("\n[Step 1/5] Pulling CRSP MSF panel with universe filters ")
panel = get_crsp_monthly_panel(db, START_DATE, END_DATE)
print(f"Rows pulled: {len(panel):,}")

print("[Step 2/5] Merging delisting returns and computing ret_eff ")
delist = get_delistings(db, START_DATE, END_DATE)
panel = add_effective_returns(panel, delist)

# 2) Build the custom Top-N indexes
print("[Step 3/5] Building Equal-, Value-, and Price-Weighted Top-N indexes ")
custom_levels = build_topn_indexes(panel, TOP_N)

# 3) ETF benchmarks (SPY, IWM, QQQ)  robust permno selection around START_DATE
print("[Step 4/5] geting ETF benchmarks ")
etf_tickers = ['SPY', 'IWM', 'QQQ']
etf_series: Dict[str, pd.Series] = {}
for tkr in etf_tickers:
    pno = pick_permno_for_ticker(db, tkr, START_TS)
    if pno is None:
        print(f"Warning: no PERMNO found for {tkr} near {START_DATE}. Skipping.")
        continue
    etf_series[tkr] = get_etf_index(db, pno, START_DATE, END_DATE, f"{tkr} (ETF)")

```

```python
# 4) Plot the results
print("[Step 5/5] Plotting and computing correlations \n")
plot_indexes(custom_df=custom_levels, etf_map=etf_series, top_n=TOP_N)

# Align and compute correlation (monthly log returns) across all available series
all_series = [custom_levels[c] for c in custom_levels.columns] + [s for s in etf_series.
    values()]
corr_mat = align_and_corr(all_series)
print("Correlation Matrix (monthly log returns):\n", corr_mat)
show_corr_table(corr_mat)
```

Listing 7: Running the Analysis

6 Visualization and Analysis

Results are plotted and the correlation matrix is rendered as a heatmap table, supporting interpretation of index performance, relative tracking, and diversification.

7 Best Practices and Further Considerations

- Validate constituent selection and effective return construction to avoid empirical bias.
- Employ annual or manageable time-windowed queries to mitigate memory issues.
- Deal cautiously with missing or suspect data before analysis.
- Test script components on small subsamples or short date ranges before scaling up.
- Clearly document all assumptions including reconstitution frequency and weighting conventions.
- Use modular functions to ensure replicability and methodological clarity for future research and analyses.
- As an extension, consider alternative reconstitution frequencies, investable universe definitions, and weighting methods.
- Another extension could include incorporating turnover, transaction costs, and stress testing for full empirical robustness.
- Close WRDS connections properly at script completion to release server resources.

8 Lessons from Custom Index Construction

The exercise of constructing custom equity indices from WRDS/CRSP data serves to practice creating financial analytics as well as to offer experiential lessons that can inform practical finance research and portfolio management. The insights learned here extend well beyond coding; they shape fundamental empirical and methodological best practices.

Data Retrieval and Cleaning from WRDS/CRSP

This project demonstrates the importance of efficient, targeted data extraction from WRDS. Query design—such as joining monthly stock files with name history tables—ensures that only relevant and investable securities are included. Imposing universe filters, like restricting to U.S. common shares listed on major exchanges, improves data quality and relevance. The iterative chunking by year represents a crucial scalability strategy when working with large datasets.

The cleaning steps, including conversion of shares outstanding to true counts and correction of price signs, highlight how raw CRSP data requires careful transformation before proceeding to analysis.

Mitigating Empirical Biases

This exercise previews two pervasive biases in empirical finance: **look-ahead bias** and **survivorship bias**. Using lagged (t-1) market caps and prices for constituent selection and weighting effectively removes look-ahead bias—reflecting the true information set available to investors at portfolio formation.

Survivorship bias is addressed by integrating delisting returns, ensuring that firms exiting the exchange do not silently vanish, skewing results positively. This nuanced treatment of delisting return data is critical for credible backtests and return measurement over longer horizons.

Reconstitution and Rebalancing Dynamics

The exercise clarifies the distinct yet intertwined concepts of reconstitution (changing the index constituents) and rebalancing (adjusting weights within a fixed universe). Monthly reconstitution coupled with appropriate rebalancing based on lagged data enforces methodological rigor. It trains students and researchers to carefully design portfolio update rules, an essential component when modeling realistic investment strategies.

Performance Evaluation and Benchmark Comparison

By constructing indexes from scratch and benchmarking them against recognized market indices like the S&P 500 (via SPY), Russell 2000 (IWM), and NASDAQ 100 (QQQ), this exercise teaches how to interpret tracking performance, style drift, and factor exposures. The comparison illuminates how weighting schemes and constituent selection can influence returns and volatility relative to well-known benchmarks.

Visualization as a Diagnostic Tool

Clear visualization of index level time series alongside reputable ETF benchmarks aids in intuitive understanding of relative performance and drawdowns. The correlation matrix heatmap of monthly log returns provides an illustrative perspective on co-movement based upon constituencies and weighting schemes.

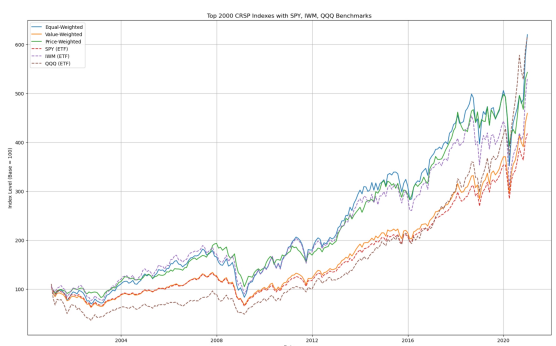

Figure 1: Return Series (2000-2020)

Figure 2: Correlation Matrix Heatmap

Pedagogical Use

And, of course, the hands-on experience gained by building indices from raw data empowers students with the practical programming and data science skills essential for modern academic and professional careers in finance and business analytics!

9 Concluding Remarks

This document provides methodological and practical implementation details for constructing custom equity indices using WRDS and CRSP data with Python. Students are encour-

aged to engage with the nuances of reconstitution and rebalancing, carefully validate their code, and extend these tools for future empirical and applied projects.

NOTES